
DAE-FAQ-Bandwidth

DAE-FAQ-Bandwidth.R5_020230709

List of Questions

Q1	How do you deal with hotspots in the Fabrix?	4
Q2	Why can't you use sequencing numbers like TCP?	4
Q3	Why can't we just use TCP and put up with its 'approximately' correct behavior?	4
Q4	The cabling appears complex, labor, parts, requires lots of patching?	5
Q5	What is the near-term goal, are we jumping into a deep chasm?	6
Q6	How many resources will this take? is it one person for 3 months, or 12, or 36?	6
Q7	What is the product?	6
Q8	What is the path to a Minimum Viable Product?	7
Q9	Do you have a specification?	7
Q10	Could Apple potentially be a customer?	7
Q11	How do you make it Legacy Compatible?	7
Q12	Why is the Topology important?	8
Q13	How will you make the protocol: Verifiable, Testable, Implementable?	8
Q14	What is the fundamental breakthrough, if it's not better bandwidth?	8
Q15	What is the Patent Strategy?	8
Q16	What is your ultimate vision for Monetization?	8
Q17	What wave in the industry are you riding?	8
Q18	What about NonStop?	8
Q19	What is the Killer Application / Use-case that you expect to pursue?	8
Q20	Who beside Apple would find FoundationDB useful?	8
Q21	How can a Transaction Fabrix have a more bandwidth than a Clos Network?	10
Q22	Why can't we just increase the bandwidth of current links going into the server?	10
Q23	What is functional Bandwidth?	13
Q24	What traffic types do you consider/account for?	13
Q25	What equation are you using to calculate the total bandwidth capacity?	13
Q26	Won't there be more disruption when links break?	13
Q27	What about Multipathing in the Clos?	13
Q28	What about Multipathing in the Transaction Fabrix ?	13
Q29	What about stranded ports?	14
Q30	What is the insight why Bandwidth is better in the Transaction Fabrix than a Clos?	14
Q31	What about QOS?	14
Q32	What other insights do you have into the scaling of bandwidth?	14
Q33	What about traffic shaping?	14
Q34	What is your any to any bisection bandwidth?	14
Q35	But what if the traffic is isotropic?	14
Q36	But you have to compare yourselves to this?	14
Q37	What is 'traffic wastage'?	15
Q38	Doesn't this require a lossless network?	15
Q39	What about the traffic that's left, isn't that still going to cause congestion?	15
Q40	Isn't QOS a solved problem in networks?	15
Q41	Why only two QOS types?	15

Q42	What if you have misconfigured, misbehaving or compromised nodes?	16
Q43	What is the distinction between a cell and a node?	16
Q44	But what if the nodes somehow get into the Sub-Virtualization Layer	16
Q45	Why don't you use high radix switches?	16
Q46	TCP Congestion Control	16
Q47	What's wrong with buffers?	16
Q48	What about Protocol Stacks?	16
Q49	What about DDoS attacks?	16
Q50	DDoS traffic is a major on-going hazard, how do you address that?	16
Q51	What are Millibottlenecks?	17
Q52	What is Mysterious Traffic?	17
Q53	What about Elephant and Mice Flows?	17
Q54	So does that mean that you don't support these traffic types?	17
Q55	What are the temporal characteristics of traditional traffic?	17
Q56	Why don't you use a centralized controller?	17
Q57	So then you have a better way of managing flows?	18
Q58	How does the Transaction Fabrix compare in Scalability?	18
Q59	What about locality?	19
Q60	What about Link utilization?	19
Q61	Where do Lambda's come into the picture?	19

Comments from the team [Will not be included in final document sent to others]

From Charlie Johnson

I'll be honest. These are tough questions on a networking scale I have a hard time dealing with. It's like how will the interstate highway system to handle all the same kinds of congestion issues, when you haven't even built a part of it to see how the roadbed holds up under big trucks.

We've been dealing with these kind of "straw-man" questions for several years now. They are easy to respond to when you come from a different perspective on the nature of causality in the network. This needs a "story" to be told, and a lot of cross-linking of the answers before we can break through the barriers of understanding. Unfortunately, that often leads to criticisms of too many words.

First, did they give you all 56 questions, and if not, which ones did they give you? Let's avoid becoming scattered.

No, I already had the FAQ-Networks done, including spreadsheets. I just added John's questions to the front of this FAQ because it was related to bandwidth. Typically people read only a subset of the answers anyway, but different folks read a different subset that appeals to them. That's why the list of questions is at the front of the document to let them pick and choose.

Second, if they did dump that entire load of questions on us, they won't be expecting a quick and probably not-well-thought-out response, so we should take our time and not get flustered.

Agreed. Let's get the benefit of the expertise on the whole team before we send this to John L + John H.

Third, at the scale of some of those questions, which will require a lot of nodes to answer, we can only use simulations to answer them and if there are problems, find a way in the simulation to address them and show the result.



And that's what we plan to do! Jonathan's Labyrinth will help us create a "digital twin" vastly faster and more easily than we were able to do previously in rust and Javascript.

Fourth, assuming John threw us a fig leaf with the scaled out KVS database project, some of the killer questions are coming from Hagerman, who like I said in my email, never liked that idea. Hagerman has dealt with some of these networking issues before on another project and he knows these problems occur at scale, so these are gotchas from him.

That's why we need to answer him with care and insight that he hasn't heard before.

As an example of responses,

1. Hot spots: we are multi path on every FPGA node in the mesh, and we also have cluster liveness to sense hotpots and route around them. Remember, there is no solution for these kinds of problems in a switching network other than dropping packets and retry storms. We never drop packets, when we get back pressure it's immediate and we reverse and go another way. Hence, we are better, how much better? Simulate that answer.

We should be able to simulate in Labyrinth why PFC hasn't worked on Ethernet, yet credit based flow control seems to work on Infiniband. Then, within that framework, show that our "reversibility" solution is better than Infiniband (which uses link-level retry).

2. TCP sequence numbers: sequence numbers are fine if you are dropping packets, we use reversibility to get the same thing.

We need to beef up our response to this. This is where the quantum inspiration comes in: In principle, we use modulo 2, 4 and 8 sequence numbers, which are guaranteed to be 'internally consistent' throughout the forward and reverse directions of the protocol interactions.

3. Complex cabling: Steve's board at the rack level and some scheme at the layer one top/bottom of rack switch.

Yes, we can point to our [Chiplet Summit](#) presentation where we showed Steve's proposal. We could also do with an article on our blog that describes this (Steve??)

4. Are we jumping in a chasm: [this is pure FUD, with no real engineering answer] Simulate and find out, then build a bridge over the chasm that is engineered to avoid the Tacoma Narrows problems. There was no way to know what the traffic on the Golden Gate would be like after they built it. Before they built it, the ferries were fine, because no one in Marin County worked in San Francisco. So, why build it? Because we don't lack the imagination of what an ultra low level mesh of FPGAs could do for scalable high interaction clusters that look like a motherboard with thousands of computing FPGAs and cores stuck in it. Now we have bridges up and down the bay and Peninsula, so I guess that first one was a good idea, even though we didn't build a wide enough one (like the Bay Bridge) and now have high speed ferries (and BART under the Bay Bridge) to increase the throughput.

You hit the nail on the head. This would be a great story to tell in our "metaphors" document.

From Chuck Sobey

The most helpful, in my opinion, is using version numbers. I suggest either

*_R1, *_R2, ... ; or the date
*_20230709; and if there are multiple version on the same day
*_20230709a, *_20230709b, ...

When multiple people are working on a draft, it is also helpful to have the date and the initials of the drafter, such as, *_20230709_CHS.

The second-most helpful, again in my opinion, is using more-descriptive titles.

Third, for documents that are shared outside of the team, we may want to have each begin with "Daedaelus_*,."

This also helps us preserve our early history more clearly – potentially useful for IP support down-the-road.



The Questions

[Q1] How do you deal with hotspots in the Fabrix?

[A1] This is a classic ([attacking the strawman](#)) argument asked by networking engineers, because: (a) it's a thorny and unresolved issue in existing networks, and (b) if someone (like us) figured out a solution, they would love to steal the idea to fix existing networks.

But existing networks can't be fixed. It's not a bug in their code or a feature they can patch – it's a flaw in their assumptions about the [the nature of time](#). This is a very deep dive which we are happy to go into.

[A1] Hotspots are a problem in conventional networking for a number of different reasons. Mostly, the receiver (or switch/forwarder) has no knowledge of what's 'incoming' on all its ports, and whether they will statistically pile-up on one output port and create (incast) congestion. For example, when networks use multi-pathing (ECMP) at the source to maximize bandwidth delivery at the destination, they do so from a God's-Eye-View¹ (GEV) perspective, which can never work. There is a large literature on this topic, of distinctly variable quality, along with debates on [hacker news](#).

[A1] By addressing hotspots with Local-Observer-View (LOV) information only, we can guarantee that buffers at the destination never overflow. Instead, we push the problem back to the senders, who remain responsible for the resource usage (buffer allocations) for their application. Our protocol goes beyond credit-based-flow-control, and guarantees "symmetric interactions" (needed by transactions), not flows (needed by video). It's a different way of thinking about things, and we can prove that it works. As the Ethernet community is fond of saying: "It doesn't work in theory, it only works in practice"².

[A1] While we can eliminate bottlenecks at the destination, we 'avoid' the undesirable behavior of switches which cause retry storms, by not going through switches for the main fabrix. The L3 boundary is at the Top of Rack Switch (TOR). Our mesh can be completely 'scale independent' inside this boundary, although in our market positioning we claim to stay within a rack (or two) for our microdatacenters. Of course, if the switch vendors adopted our protocol, they can also benefit from this breakthrough. But we don't require them to do that, any more than VMWare needed Microsoft's permission to revolutionize the operating systems world.

[Q2] Why can't you use sequencing numbers like TCP?

[A2] We do (partially) use sequence numbers; but with three extremely important caveats:

1. These sequencing numbers must be modulo counters (of some small base) and *reversible* to account for errors and overflows gracefully – instead of abruptly, and losing information needed to make data structures consistent.
2. They must also be 'bounded'. Any 'variable length' buffer is provably incapable of being 'atomic'. These 'interaction buffers' must be 'pairs' of 2, 4 (or any multiple of 4) 'interactions'. This comes out of the mathematical models³. Anything outside of this model we are incapable of proving are atomic. Anything in this finite range of sequences, is provably atomic. This is a big deal in transaction systems.
3. Our sequence numbers are complex numbers (in the mathematical sense of $\sqrt{-1}$).

[A2] We know this goes against the grain of conventional thinking in networking, but those ingrained beliefs come from half a lifetime of optimizing for bandwidth. When reliability, low latency, and security are far more important in today's distributed microservices applications.

[Q3] Why can't we just use TCP and put up with its' 'approximately' correct behavior?

[A3] [TCP's Death has been predicted](#). Here's the abstract of John Ousterhout's Paper [It's Time to Replace TCP in the Datacenter](#):

¹Has nothing to do with religion, and everything to do with programmers trying to act like Göd from their perspective.

²Conversation with Bob Metcalfe, July 2022 at the Wolfram Summer School.

³And is compatible with Larry Peterson's Request-Response RPC approach.

“In spite of its long and successful history, TCP is a poor transport protocol for modern datacenters. Every significant element of TCP, from its stream orientation to its expectation of in-order packet delivery, is wrong for the datacenter. It is time to recognize that TCP’s problems are too fundamental and interrelated to be fixed; the only way to harness the full performance potential of modern networks is to introduce a new transport protocol into the datacenter. Homa demonstrates that it is possible to create a transport protocol that avoids all of TCP’s problems. Although Homa is not API-compatible with TCP, it should be possible to bring it into widespread usage by integrating it with RPC frameworks.”

[A3] John Ousterhout believes that TCP (L3) needs to be replaced, and proposes [Homa](#).

[A3] Larry Peterson thinks [It’s Groundhog Day on the debate in TCP](#), and proposes using ‘paired’ ([request-response RPC protocols](#)) instead of TCP.

[A3] From our perspective, both John and Larry are correct: Datacenters (not just the Internet) seems to be “missing a standard protocol for the request/response paradigm”. We believe the problem goes a little deeper, and we need to enhance L2 as well, because it’s easier to keep heartbeats going across a single link than it is across an entire Clos network. This ties in directly to our intuition about Quantum Information Theory, and Multiway Systems. The magic of making this work at scale a mathematical technique called ‘compositionality’.

[A3] Sequencing numbers are a ‘Forward in Time Only’ (FITO) concept. Our protocols include the counterfactual⁴ (in our protocol, sequence numbers can go backwards), and incorporate a finite partial order set (in both directions) to manage the operations on data structures, to guarantee they will not be corrupted, provided both the forward and reverse sequencing is ‘successive’. Monotonicity is neither necessary nor sufficient. The concept of monotonicity keeps us stuck in the FITO world, and prevents us seeing the more general solution.

[A3] Larry Peterson provides a nice description of the request/response paradigm (as an alternative to TCP) and how it relates to ‘exactly once’ semantics in [Chapter 5.3](#) of his book *Computer Networks: A Systems Approach*:

RPC reliability may include the property known as at-most-once semantics. This means that for every request message that the client sends, at most one copy of that message is delivered to the server. Each time the client calls a remote procedure, that procedure is invoked at most one time on the server machine. We say “at most once” rather than “exactly once” because it is always possible that either the network or the server machine has failed, making it impossible to deliver even one copy of the request message.

[Q4] The cabling appears complex, labor, parts, requires lots of patching?

[A4] We take an opposite view. Direct connections are *simpler*, because they represent a 2D *planar* topology which we can more directly discover their topology and evolving relationships.

[A4] Within a single rack, using today’s off the shelf components, is possible to prove our reliable protocol value proposition. However, existing cabling (copper, and especially Fiber) is unnecessarily expensive. We can use the John Lockwood idea of L1 switches to make customers feel comfortable with the idea of legacy compatibility (and we should), but this would be an illusion, which can be used to disrupt the conventional networking industry with something far simpler, more reliable and more programmable.

[A4] Our more important goal is the Chiplet Revolution. Where we anticipate riding the wave of innovation for [Chiplet technologies](#), both in the Interconnect (bunch of wires) and new motherboard technologies, where 2D arrays of chipsets coexist each with their own smartNICs, and natural 8-valency connectivity.

⁴See Chiara Marletto’s book: [the Science of Can and Can’t](#).

[Q5] What is the near-term goal, are we jumping into a deep chasm?

[A5] Our investors, customers and collaborators, will expect us to de-risk our technology before making any product claims. Extraordinary claims require extraordinary evidence. As with every startup, we must avoid trying to boil the ocean.

[Q6] How many resources will this take? is it one person for 3 months, or 12, or 36?

[A6] We originally (last year) figured out that the ‘minimum viable product’ would take around 25-man months if Paul did it all himself. This includes learning the FPGA toolchain, and continuing to build his skills in Rust. But then he could get nothing else done, and the customer relationships, marketing, operations and fundraising would go by the wayside.

If Charlie did the MVP all by himself (at half time, i.e. nights and weekends) we imagined it would take him around 6-8 months because of the skills he already had?

However, with help from other engineers (e.g. folks at Algo Logic, Sahas, Tyler) this could be sped up to maybe 3 months? We are checking with Charlie now to see if this is still a reasonable estimate? (Question for Charlie: When do you think we can achieve the next milestone for the improved demo?)

We now have Sahas on board who can start helping Charlie do some of the FPGA work.

Jonathan will be working higher up the stack, on the interface between Labyrinth, Lingua Franca, and the simulator. His formal verification tools will provide a much faster and more efficient toolset for us to do development toward our vision for an “App-Store for Distributed Applications”.

We are currently trying to recruit engineers to work higher up in the toolchain, to help deploy FoundationDB using Kubernetes. Sahas could possibly take on this role.

We need to get past the threshold where we have a minimum viable product in our sights, and then we can go out fundraising.

The original plan was to start fundraising on September 1st, and to close by Thanksgiving. Jonathan and Paul decided a couple of weeks ago that there should be no reason why we can’t start fundraising sooner. As long as we have the pieces in place (Demo, Business Plan, Engineering Plan, Customer Interviews).

The main reason we need more than 5-7 people is because we need to offload from them all the administrative, legal and organizational (collaborator/customer) meetings in order for them to focus 100% of their time on creating the product. This goes especially for Paul, Charlie, Jonathan, and Sahas.

This is why we need investment. We can make progress incrementally with a few people (some half time, some full time), but it takes a team of complimentary expertise in hardware, software and architecture/simulation to do so in a reasonable time to market – which this opportunity demands. Plus a complimentary team of marketers, salespeople and administrative/legal to stand up a sustainable company.

[Q7] What is the product?

[A7] A new Infrastructure Foundation for High-reliability, Transactional Key Value Stores for Distributed Applications. We become the ‘underlay’ for all transactional based systems. ‘We make Transactions reliable’.

[A7] Note: Algo-Logic implemented a KVS base node in FPGA that achieved sub-microsecond latency to read data from memory and a throughput of 150M reads/second per instance. Scaling out the KVS to a distributed database like FoundationDB could be a killer application. Redis, Snowflake and CouchDB have certainly had success. [Algo Logic KVS 1](#) [Algo Logic KVS 2](#)

[A7] Our stack looks as follows: [Include Graphic]

- Customer Applications (Apple, CouchDB, Snowflake)
- FoundationDB [[Paper](#), [Website](#), [GitHub](#)]
- Lingua Franca [[Presentation](#), [Community](#). GitHub.
- Labyrinth Distributed Systems API’s
- Labyrinth Distributed Systems Functions in the SmartNICs



[Q8] What is the path to a Minimum Viable Product?

[A8] The sequence to our minimum viable product is:

1. 2-Node bipartite link protocol and reversibility token. [DONE]
2. 3-Node Triangle Demo. Event harvesting in the face of packet loss on one link.
3. 4-Node Demo. Demonstrate Liveness Tensor: Awareness of link failure more than 1 hop away.
4. 9-Node Demo. Demonstrate Liveness Tensor + Cluster resilience for FoundationDB
5. 25-Node Demo. Demonstrate 2-hop hypertile. Enterprise class NonStop Clusters.

[A8] The above steps are milestones for the three key technologies which comprise our breakthrough's:

- Reversible single link protocol
- Cluster Liveness 2 or 3 hop awareness of broken link status.
- Hypertile Failover Clusters.

[Q9] Do you have a specification?

[A9] Yes. We have a modular specification, Which includes the following documents (available on request):

- Slice-Engine-Specification.pdf.
- Liveness-Tensor.pdf
- Reversible-State-Machine.pdf
- DAE.pdf (450 pages including references).

[Q10] Could Apple potentially be a customer?

[A10] Yes. They have been my eventual target for a long time (at least the last 5 years). It would be foolish to go after Apple when we are not ready. We get one silver bullet, through the connections I know, to get a hearing. Apple's position is that they are too busy and overloaded to look at startups, unless they already have something working and have already "de-risked" the technology. Unfortunately, Apple's policy is also to "acquire" companies early. So they can get them before they are worth a lot.

This is what happened to FoundationDB. They already had it working, had an open-source user base, and had proved the technology in several demonstrations on youtube and at conferences.

[Q11] How do you make it Legacy Compatible?

[A11] There are three answers to the is question:

1. Like VMWare, we make it compatible at a well defined interface abstraction layer (e.g. Virtual Machines did it at the x86 instruction set, Java did it with JVM byte code, we do it at the socket later).
2. We don't need to be compatible with every layer of the network protocols, only with the applications. Customers want another 10 years of life for their applications, not another 10 years of employment for their Cisco certified engineers.
3. The only thing we make incompatible, is the attack surface presented by mixing L2 and L3 protocols on the same wires.

[Q12] Why is the Topology important?

[A12] Because thinking in terms of radix misses the half the picture. This LOV constraint, has a rich literature. Our breakthrough is distinguishing the LOV from the GEV.

[Q13] How will you make the protocol: Verifiable, Testable, Implementable?

[A13] This is where our protocol framework comes in: Labyrinth compliments the FoundationDB methodology of ... (I'm hoping someone on the team is reading this far, and has read the paper).

[Q14] What is the fundamental breakthrough, if it's not better bandwidth?

[Q15] What is the Patent Strategy?

[A15] We plan to share, make open-source, and license (reasonably and fairly) our IP. We have already broadcasted our intentions in our [announcements](#).

[A15] Loosely speaking, we intend to make everything running in the host processor (software infrastructure) open source (freemium), and everything in the hardware FPGA's proprietary. Moderated by an open-core policy that allows academic institutions to freely use the system (GitHub repository), and for commercial organizations to benefit from our technology through licensing system.

[Q16] What is your ultimate vision for Monetization?

[A16] We will become the 'App Store' for distributed applications. Our ecosystem will guarantee not only the reliability of transactions among distributed systems, but their security also.

[Q17] What wave in the industry are you riding?

[A17] We believe our technology is applicable on the edge, with transaction-based, AI/ML, Digital Twins, and Interface to Quantum Computing.

[A17] The SmartNIC Revolution in particular is a well defined market, see email by [Dell'Oro Group](#).

[Q18] What about NonStop?

[A18] Current systems are severely compromised by failures in network protocols. Nonstop won the day by not corrupting data structures (logs), even though they had to operate in the highly adversarial environment of packet drops, duplication and reordering in the network.

[Q19] What is the Killer Application / Use-case that you expect to pursue?

[A19] For any startup the key question is how do we resolve the uncertainty, of whether our technology is relevant or not? The answer is you talk to customers. Who, through the expression of their pain points, will enable us to focus on the 3 (out of 100) possibilities that our development team need to focus on.

[Q20] Who beside Apple would find FoundationDB useful?

[A20] Anyone who uses Snowflake or FoundationDB.

[A20] in 2015, Apple acquired [FoundationDB](#). It had become obvious that both their Oracle-based transaction systems, and their Cassandra-based file storage systems were inadequate. FoundationDB took a different approach: they first created a simulator to verify their concepts and protocols, and only then did they build their production system.

[A20] Despite the enormous effort and discipline that went into the FoundationDB design and deployment, their still remain several theoretical, and practical issues related to their implementation, We solve both. By running

[A20] First of all, Apple is a potential customer. We won't get to approach them unless we have something working.

[A20] Second, the entire Snowflake and CouchDB environment (primers and customers, are highly visible to us and could also be prospective customers.

One of the reasons for its popularity and growing open source community is FoundationDB's focus on the "lower half" of a database, leaving the rest to its 'layers' – stateless applications developed on top to provide various data models and other capabilities. With this, applications that would traditionally require completely different types of storage systems, can instead all leverage FDB. Indeed, the wide range of layers that have been built on FDB in recent years is evidence of the usefulness of this unusual design. For example, the FoundationDB Record Layer¹⁰ adds back much of what users expect from a relational database, and JanusGraph,⁶ a graph database, provides an implementation as a FoundationDB layer.⁵ In its newest release, CouchDB (arguably the first NoSQL system) is being rebuilt as a layer on top of FoundationDB.

===== End of Direct Questions from John Lockwood and John Hagerman =====

[Q21] How can a Transaction Fabrix have a more bandwidth than a Clos Network?

[A21] In a nutshell (assuming 25GbE for all ports): (under the same assumption of 25Gbit ports):

There are more ports coming out of the server, and (2) those links carry mostly functional bandwidth (which uses only 1 link per functional packet), rather than a Clos network, where the switched fabrix carries redundant (transit) copies of the same functional packets.

- Each cell in the Fabrix has 8 ports for *independent* links. Thats 200 Gbits/s into the fabrix, *per cell*.
 - Clos has 2-4 ports. That's 50 - 100 Gbits/s node bandwidth into the network, *per server*.
- The majority of Fabrix Link's carry *functional* packets (each packet appears only once on *one* link).
 - Clos links carry multiple (transit) copies of the same packets, *plus many types of wasted traffic*.
- The Transaction Fabrix enables agents (shared data + computational foci) to migrate and self-organize.
 - Functions 1 hop away in an application graph migrate to become adjacent in the physical fabrix.

[Q22] Why can't we just increase the bandwidth of current links going into the server?

[A22] Because above 40Gbps, we start to see [Dark Packets](#), due to the gap between cache and DRAM latency (which has stagnated at 100ns per access). At 100GbE almost all packet sizes are under the latency line. So we can't get more bandwidth in and out of a server unless we go to more ports. Cells in the Transaction Fabrix have 8+ ports. By operating each port in the range 25Gbps, we maximize the total throughput of the server (assuming more cores are available), or we increase the ability fo the Cells to be forwarding services via the NIC (i.e. packets don't need to come in and go out of main memory and suffer the 100ns per access penalty when being forwarded).

[A22] Communication topologies for modern microservice applications in a datacenter are *not* flat, they are *graphs*. Funneling traffic from all of these servers through fewer links inserts both an unnecessary choke point, and a SPOF (Single Point of Failure). Imposing a flat A2A (Any to Any) network on the datacenter is no longer appropriate when most datacenter traffic is East-West. A flat network throws away important topology (adjacency) information that the provisioning software for the application graph can exploit to enable provisioning to be fully automatic (without the need for human intervention).

[A22] The analysis in the table (next page) shows a datacenter row of 20 racks of 42 cells each⁵. We calculate the bandwidth as the total number of 'usable' links (those which carry only functional bandwidth). For ~800 cells (servers in the Clos), a Transaction Fabrix can achieve over 600 Terabits/s, whereas the 2-slice Clos network can achieve only 80 Terabits/s with the same assumptions, and a quad redundant (4-slice Clos) is 80 Terabits/s. The figure below shows just the link bandwidth differences between the Transaction Fabrix and a Clos, not taking account of the difference between transit, functional and wasted bandwidth:

800 Cells/Servers

~600Tb/s for CF

~80 Tb/s for Clos

Cellular fabrix

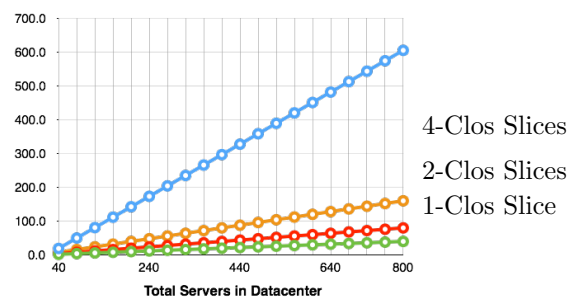
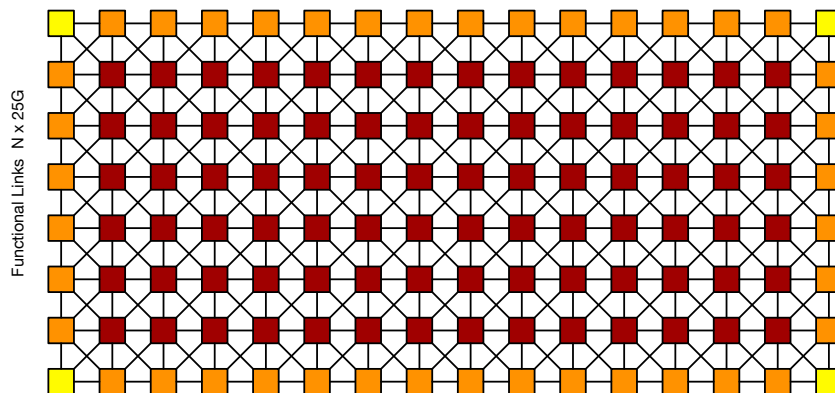


Figure 1: Functional Bandwidth

⁵Assumes a conventional configuration of 42U Racks & 1U servers. Many other configurations are possible. Such as 2U, 3U, 4U+ (or fractional U) server height, and/or multiple (e.g. 2, 3, 4+) cells across the width of the rack.

Cellular Fabric/Clos Bandwidth Comparison

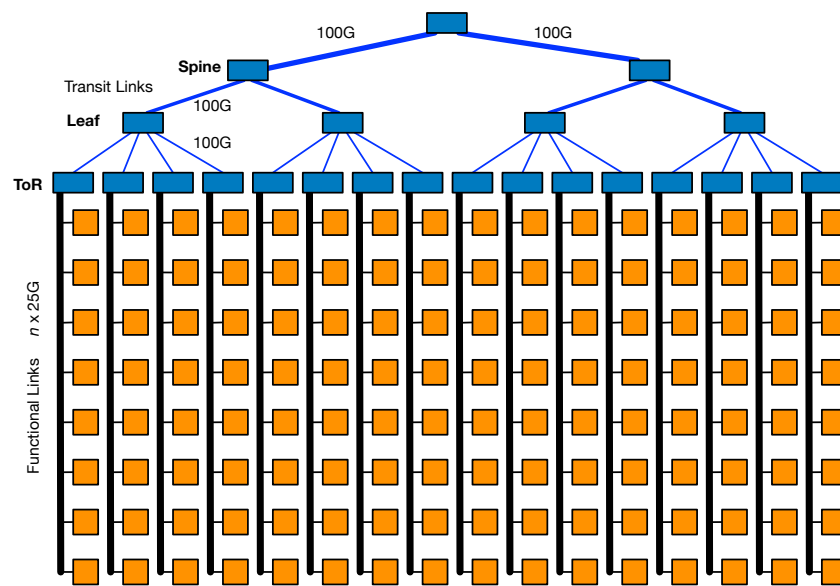
Servers Per rack n	Racks per Row m	Total Servers $n*m$	Total Cellular Fabric BW 8-port cells (Fraction packets Functional)	Clos 2-slice Clos BW (2-port servers)	Clos 4-slice Clos BW (4-port servers)	Clos 8-slice Clos BW (8-port servers)
	20		1			
2	20	40	19.2	2.0	4.0	8.0
4	20	80	50.0	4.0	8.0	16.0
6	20	120	80.8	6.0	12.0	24.0
8	20	160	111.6	8.0	16.0	32.0
10	20	200	142.4	10.0	20.0	40.0
12	20	240	173.2	12.0	24.0	48.0
14	20	280	204.0	14.0	28.0	56.0
16	20	320	234.8	16.0	32.0	64.0
18	20	360	265.6	18.0	36.0	72.0
20	20	400	296.4	20.0	40.0	80.0
22	20	440	327.2	22.0	44.0	88.0
24	20	480	358.0	24.0	48.0	96.0
26	20	520	388.8	26.0	52.0	104.0
28	20	560	419.6	28.0	56.0	112.0
30	20	600	450.4	30.0	60.0	120.0
32	20	640	481.2	32.0	64.0	128.0
34	20	680	512.0	34.0	68.0	136.0
36	20	720	542.8	36.0	72.0	144.0
38	20	760	573.6	38.0	76.0	152.0
40	20	800	604.4	40.0	80.0	160.0
42	20	840	635.2			



Cellular Fabric

Clos Functional Bandwidth

Servers Per rack* <i>n</i>	Racks per Row <i>m</i>	Total Servers <i>n*m</i>	Func. Links <i>m ToRs</i>	Total Func. Link BW (GBits)*	Total Bandwidth required by servers (Per Clos Slice) Tbits (= 1000 Gbits) Server to ToR BW only			
Bandwidth	20			12.5	1	2	3	4
2	20	40	40	500	0.5	1	1.5	2
4	20	80	80	1,000	1	2	3	4
6	20	120	120	1,500	1.5	3	4.5	6
8	20	160	160	2,000	2	4	6	8
10	20	200	200	2,500	2.5	5	7.5	10
12	20	240	240	3,000	3	6	9	12
14	20	280	280	3,500	3.5	7	10.5	14
16	20	320	320	4,000	4	8	12	16
18	20	360	360	4,500	4.5	9	13.5	18
20	20	400	400	5,000	5	10	15	20
22	20	440	440	5,500	5.5	11	16.5	22
24	20	480	480	6,000	6	12	18	24
26	20	520	520	6,500	6.5	13	19.5	26
28	20	560	560	7,000	7	14	21	28
30	20	600	600	7,500	7.5	15	22.5	30
32	20	640	640	8,000	8	16	24	32
34	20	680	680	8,500	8.5	17	25.5	34
36	20	720	720	9,000	9	18	27	36
38	20	760	760	9,500	9.5	19	28.5	38
40	20	800	800	10,000	10	20	30	40
42	20	N/A	N/A — (ToR Switches take up this space in the Rack, in a Clos)					



[Q23] What is functional Bandwidth?

[A23] If we are modeling network bandwidth, you should keep track of the non-traffic (goodput). Clos networks have redundant (transit) traffic flowing over the links. When we calculate for total link bandwidth, the links between the switch layers don't count because they carry only transit, and not functional bandwidth. This becomes even more important when we look at control traffic, adversarial traffic, and timeout storms⁶.

[A23] The only thing that's important is functional bandwidth, i.e. The amount of bandwidth going into and out of the servers. In a Clos Network, every functional link must be formed from a minimum of two physical links and one switch. So the actual functional bandwidth is half the number of functional links, *even when communication is between two servers in the same rack* – they have to go up and down through the TOR switch.

[Q24] What traffic types do you consider/account for?

[A24] The basic types are:

Storms Timeout storms, gossip storms, reconstruction [ref Facebook].

Mysterious Traffic that shouldn't be there but is. (Adversarial?).

Heartbeats Conventional and gossip.

Transit Each packet traverses multiple links in the Clos.

[Q25] What equation are you using to calculate the total bandwidth capacity?

[A25] The basic equation for an apples for apples comparison is to compare a 4 slice Clos (4 ToRs per rack, 4 ports into each server), and the 8 port cellular design. Assume all Links to ToRs, and inter-cell links are 25Gbit over CAT8 cabling. We are not even distinguishing, at this stage, between the functional traffic, wasted traffic and adversarial traffic.

[A25] Simplistically speaking, there's an 8:4 or, a 2:1 advantage for the Transaction Fabrix simply because there are 2× or 4× the functional links in a cell than there are in a server in a Clos. One key metric assumes that every functional link carries independent traffic, and not transit traffic. The advantage is less if cells start becoming switches, but this is less probable than might be thought (see FAQ:Latency).

[Q26] Won't there be more disruption when links break?

[A26] On the contrary. The higher you go in a Clos network the more servers share a given switch and therefore the more disruptive will be a routing reconfiguration. Whereas even a stochastic distribution of packets can reserve a path (tree).

[Q27] What about Multipathing in the Clos?

[A27] The maximum bandwidth calculated above assumes that the server can drive all its ports together. This is not necessary; while the Clos network goes to great lengths to introduce multipathing (e.g. [ECMP](#)) the Transaction Fabrix has this mechanism intrinsically built-in. For example, if cells/servers have only enough CPU and memory bandwidth to support say 100 Mbits/total bandwidth, the Cell can select one of 8 ports to send data to its destination(s), whereas the Clos server has a choice of only 2-4, and expects the switched network to perform the necessary load balancing across the Clos network.

[Q28] What about Multipathing in the Transaction Fabrix ?

⁶During packet storms, most of the spaces on the network buffers are filled with packets. Modelling each packet takes up an enormous amount of memory. Keeping track of the empty spaces instead would use require almost no memory. Furthermore, the dynamics of the non-packets might be more amenable to analysis. (See [Traffic Analysis](#)). A concept from information theory: minimum description length. Fewer words are required to say 'No-L' than to list 'ABCDEFGHIJKLMNOPQRSTUVWXYZ.'

[A28] We have a more available paths without a global reorganization; almost all rerouting is local; i.e. between adjacent cells one hop away.

[Q29] What about stranded ports?

[A29] Unlike a Clos, there are no stranded ports in a Transaction Fabrix . Any remaining open ports on any cell cell may simply be connected to any open port on any other cell, to reduce average hopcount, or to improve resilience and bandwidth.

[Q30] What is the insight why Bandwidth is better in the Transaction Fabrix than a Clos?

[A30] Our rich interconnect means there are many paths between any two nodes, so: (1) single-flow bandwidth is higher, and (2) Trunk (bonding/multipath) bandwidth can be higher. Our bandwidth may also be better because each link is loaded less.

[A30] We also eliminate many types of wasted duplicated and adversarial traffic. See ‘Traffic wastage’.

[Q31] What about QOS?

[A31] The Clos world is biased toward pre-determining QOS (and it doesn’t work anyway). In our world, QOS is derived from the allocation of processes to TRAPH’s (TRee-grAPHS) in configurations that continually adapt to their local environments without needing a GEV (God’s-Eye View) central controller.

[Q32] What other insights do you have into the scaling of bandwidth?

[A32] As link speeds improve, the interpacket gap decreases. 40GbE represents the first time where several small packet sizes are just below DRAM latency. At 100GbE almost all packet sizes are under the latency line. This represents a new regime for the future where bottleneck is now the memory and cache hierarchy. If a system takes longer to service a DRAM miss than it does to obtain a new packet packets will drop. This forces network operators to run next generation NICs at current generation speeds and increases system cost as the only way around this problem is to increase core count. [See: [Dark Packets](#)]

[Q33] What about traffic shaping?

[A33] Traffic shaping is a far less significant issue in the Transaction Fabrix . Our dense fabrix provides many more disjoint paths – [Menger’s theorem is our friend](#) [See [FAQ-Resilience](#)].

[Q34] What is your any to any bisection bandwidth?

[A34] That question is only relevant to a Clos, where all the traffic has to go through the choke points of a switched network. The graph on the very first page shows a new perspective: every link in the Transaction Fabrix is capable of carrying functional bandwidth, i.e. traffic that is part of the computation, and not just transiting through the switched network.

[A34] Previous datacenter network architectures attempt to maximize bisection bandwidth. While those approaches may be well suited for data centers, which run applications that stress the network’s fabrix with all-to-all (A2A) traffic, this would be unwarranted in data centers where the bisection bandwidth is not taxed by the applications. [[Benson Et al.](#)]. Bisection bandwidth is a misleading metric, and a wholly inappropriate concept. It is this very distinction where the Transaction Fabrix will provide significant advantages.

[Q35] But what if the traffic is isotropic?

[A35] If by isotropic (the same in every direction) you mean isotropic? we are exactly the opposite of isotropic. this is one of our fundamental breakthroughs. This is the precise distinction that enables us to do what conventional networks cannot.

[Q36] But you have to compare yourselves to this?



[A36] We are trying to get you to think in a bigger box.

[Q37] What is ‘traffic wastage’?

[A37] There are many kinds of ‘traffic’ that waste bandwidth in a modern datacenter:

- Cluster Heartbeats, which scales as N^2 .
- Retries, when packets get lost. Timeouts go off and duplicate or triplicate traffic is resent.
- This leads to retry storms (for which the network has to be sized for).
- Gossip: redundant traffic to overcome the lack of reliable failure detectors.
- This leads to gossip storms (for which the network has to be sized for).
- Erasure coded traffic. A single packet coming in is sharded across many packets for blocks.
- This leads to reconstruction storms, when a shard appears to be absent.
- Mysterious traffic: which shouldn’t be there but is.

We eliminate all of the above. The traffic that’s left is the functional traffic, i.e. that traffic which contributes to the computational work of the datacenter.

[Q38] Doesn’t this require a lossless network?

[A38] We have a way of marking packets as lossless. A conventional Ethernet switched network does not.

[Q39] What about the traffic that’s left, isn’t that still going to cause congestion?

[A39] If you mean DDoS traffic, we eliminate that at the boundary of our fabric. Internal nodes are not addressable by attackers on the outside.

[A39] If you mean Multicast or Any-cast Traffic, we eliminate that by having only Treecast, where each packet traverses each needed link only once.

[A39] If you mean load balancing traffic, we subsume that by equipping our links with node availability and decorating them with load status. Load balancing is built into the foundation of the tree mechanism.

[A39] If you mean monitoring and logging traffic, we use the remaining lightly loaded trees to carry this traffic off the main paths of the computation fabric.

[Q40] Isn’t QOS a solved problem in networks?

[A40] QOS is an complex byzantine mess that doesn’t solve the problem. There are only two kinds of packets: (1) ENTL and ENTT. (2) Everything else. ENTL and ENTT are extremely simple and well-behaved⁷. ENTL works only between two cells, to maintain liveness. ENTT works across a chain or a tree with single *fixed size packets*. Type (1) packets have their own buffer SINGLE PACKET Size buffer. Type (2) packets include everything else. We don’t trust them to have the right properties to solve distributed systems problems, so they go second.

[Q41] Why only two QOS types?

[A41] Because its more important to drop large packets than to drop small packets. This is a direct resilience-performance and resilience-security tradeoff we refuse to make. Safety (resilience and security) are more important than performance. Traditional switched networks disproportionately drop the ACK packets⁸ which makes things far worse (now we have a transaction in progress that can’t finish). We would rather have a transaction that has a hard time starting than one that has a hard time finishing.

⁷Ask us about zero, one and ∞ – type 2 packets belong in the latter

⁸See: [Availability in Globally Distributed Storage Systems](#)

[Q42] What if you have misconfigured, misbehaving or compromised nodes?

[A42] Cells are not nodes in the conventional sense. Each cell has an invisible Sub-Virtualization Layer that creates, segregates and maintains the fabric. All that applications (or the Linux operating system for that matter) can see, are other nodes that belong in the same tenant. See FAQ-Security.

[A42] Not only is wasted traffic eliminated as described above, so are many kinds of adversarial traffic. Each tenant can DDoS itself, but not other tenants in other TRAPHs.

[Q43] What is the distinction between a cell and a node?

[A43] A node is anything you (the customer) wants it to be. A cell includes our Sub-Virtualization Layer, which protects the fabric from Byzantine behavior of any of the nodes.

[Q44] But what if the nodes somehow get into the Sub-Virtualization Layer

[A44] We know of no current attack vector capable of doing this. We also have countermeasures for any nefarious behavior by the Intel Management Engine.

[Q45] Why don't you use high radix switches?

[A45] The biggest single problem with high radix switches is their blast radius; not simply because of hard failures; but for the soft failures: timeout storms they cause by dropping, re-ordering, delaying and duplicating packets. (Switches are DReDDful).

[A45] The Goldilocks radix is 8, not 128. That's why we call it 'valency', so people don't get confused thinking that more is better.

[Q46] TCP Congestion Control

[A46] [Mark Handley Video](#).

[A46] As the per-flow product of bandwidth and latency increases, TCP becomes inefficient and prone to instability, regardless of the queuing scheme. [[Katabi et al, Congestion Control for High Bandwidth-Delay Product Networks](#).]

[Q47] What's wrong with buffers?

[A47] Bufferbloat: the undesirable latency that comes from a router or other network equipment buffering too much data. In a transactional system, buffering is the enemy. It increases latency, and diminishes transactional performance and reliability.

[Q48] What about Protocol Stacks?

[A48] See: [Re-architecting datacenter networks and stacks for low latency and high performance](#)

[A48] See: [ixy - a userspace network driver in 1000 lines of code](#) ixy is a simple userspace packet processing framework. It takes exclusive control of a network adapter and implements the whole driver in userspace. Its architecture is similar to DPDK and Snabb and completely different from (seemingly similar) frameworks such as netmap, pfq, pf_ring, or XDP (all of which rely on kernel components).

[A48] See: [User Space Network Drivers](#) by Paul Emmerich et. al.

[Q49] What about DDoS attacks?

[Q50] DDoS traffic is a major on-going hazard, how do you address that?

Short packets are dropped disproportionately... ⁹

⁹See [Shan et al. Tail Attacks on Web Applications](#).

[A50] Reflection attacks continued to hound defenders. In prior quarters, DNS, NTP, and CHARGEN were the top three reflection attack vectors. [[Akami State of the Internet](#)] / [security report](#).]

[Q51] What are Millibottlenecks?

[A51] [Tail attacks on web applications](#). DDoS attacks on classic N-tier web applications designed to push tail latency high while simultaneously being very hard to detect using traditional monitoring tools. ‘Millibottlenecks’ (on CPU or I/O) can be caused by filling up buffer using short bursts of traffic. This may be byzantine (any unknown cause) or deliberate by an attacker sending timed pulses of traffic.

[Q52] What is Mysterious Traffic?

[A52] In any datacenter network where an anonymous process on a node may send traffic to endpoints through the Clos Network, there will be ‘mysterious’ traffic. No one in the developer community, or the tenants will admit to owning this traffic, yet the network operator cannot block this traffic without fear of causing something important to fail. Attackers use this blind spot to create mischief as they masquerade as legitimate processes on machines.

[Q53] What about Elephant and Mice Flows?

[A53] While elephants may have flows, mice have ‘*interactions*’. The easy measure to focus on is total bandwidth, where it is extremely attractive to solve the problem in intuitive (but incorrect) ways, such as larger packets and larger buffers.

[A53] The Transaction Fabrix is designed for high ‘temporal intimacy’ (mutual state coupling) in the core. These properties are extremely important not only for responsiveness, but for transactional integrity. We therefore, (for the time being) ban all mechanisms in the core that seek to optimize high volume single flows, such as video.

[Q54] So does that mean that you don’t support these traffic types?

[A54] We do support those traffic types, but only on the edge, where we interface to conventional datacenter protocols (TCP/IP). In the core, we deprecate premature optimizations that counter the key goals of the system: Simplicity, Resilience and Security.

[A54] The goal of the underlying communication substrate is to transfer complete, atomic and fixed-size packets. Anything else would be premature optimization. Variable length messages would violate the principle of simplicity, by creating cascading complexities in how to mitigate partial transfers¹⁰.

[Q55] What are the temporal characteristics of traditional traffic?

[A55] The temporal characteristics of packet traces are explored in [[Benson Et al. Network Traffic Characteristics of Data Centers in the Wild](#)].

[Q56] Why don’t you use a centralized controller?

[A56] Many existing datacenters rely in some form or another on a centralized controller for configuring routes or for disseminating routing information to endhosts. A centralized controller is only practical if it is able to scale up to meet the demands of the traffic characteristics within the data centers. For distributed controllers (e.g. OpenFlow), flow tables are a constrained resource on today’s switches. There is no such

¹⁰See [Some Constraints and Tradeoffs in the Design of Network Communications](#) This paper illustrates the underlying concern with arbitrary length messages (what they call partial transfers). To overcome these problems they suggest: to ban partial transfers to processes with well-known ports. This is the approach taken in ARPANET, where communication to well known ports are restricted to short complete messages [9] which are used to set up a separate connection for subsequent communication?. We enable this separate communication as a separate tree, dedicated to providing legacy communication (e.g. IP – so that the legacy application TCP stack has to handle the segmentation and reassembly). This substantially simpler approach eliminates unnecessary complexity in our Network Asset Layer (NAL).

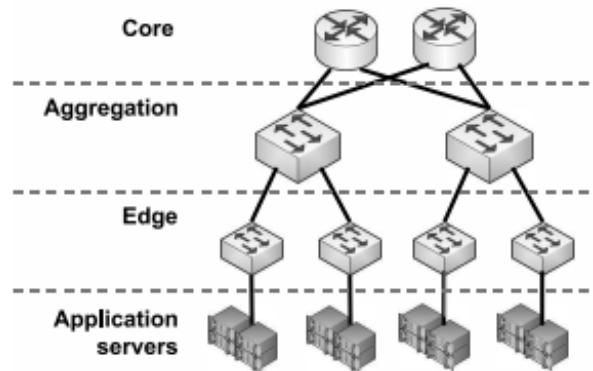


Figure 1: Canonical 3-Tier data center topology.

constraint in the Transaction Fabrix because cells have access to a relatively large amount of additional resources should they need it.

[Q57] So then you have a better way of managing flows?

[A57] Flows are a specific concept for CDC's. We support a more generalized concept called *Treez*. Conventional flows can be mapped to a Tree, but the tree can do more: (a) **treecast** guarantees the delivery to a specific number of endpoints (the full set, or a subset, such as 3,7,9 cellpoints). (b) Notifications on the departure of cells (agents) for that tree, and the arrival of new ones.

[Q58] How does the Transaction Fabrix compare in Scalability?

[A58] There are significant challenges to scalability in conventional networks [see: [3 Reasons Why Networking Breaks HCI Scalability](#)] "Networking that node into the cluster can create a lot of cost and complexity that breaks the HCI scalability pitch.":

Network Provisioning When adding a node, actually after racking that node, it is necessary to connect it to the network. In most cases, it requires making several connections. Most IT planners will have at a minimum, a VLAN dedicated to storage I/O traffic, a VLAN dedicated to east-west cluster traffic, a connection to the corporate network, and a connection dedicated to cluster management. All of these connections need ports from the top of rack switch, provisioned and configured for use. In many cases, the team in charge of the HCI environment does not have the necessary credentials and skills to perform the needed network provisioning. Many data centers report that it takes days to provision networking to the node after its physical installation in the rack.

Network Performance HCI is a shared everything environment which make troubleshooting more difficult. Isolating which VM is consuming what network resources is very difficult since the network hardware and network management is a separate silo from the management of the HCI environment. IT is typically caught off guard when a performance problem occurs and has limited tools to isolate networking problems down to a virtual machine level.

Networking Costs One of the advantages of the ?just add a node? concept is the makeup of the node itself. The node is a server, typically made from commodity hardware. The storage for the environment is in that node, using server class drives, that are less expensive than the enterprise class drives found in shared SAN/NAS storage systems. As a result, IT can add additional compute, storage performance and storage capacity, to the cluster at very competitive price points.

"The cost of networking is the same whether the organization uses HCI or traditional SAN/NAS. HCI can be more expensive because it consumes more ports, more top of rack switches and more mid and core

level Clos switches. A Transaction Fabrix grows in the number of ports in proportion of the number of cell: with the CPU cores, total memory, and

[Q59] What about locality?

[A59] In cloud data centers, a significant fraction of traffic stays inside the rack, while the opposite is true for enterprise and campus data centers. While the mechanisms to colocate applications within the same rack exist, they have a reputation for requiring human attention, being complex to understand, and are relatively static. For the Transaction Fabrix locality is expressly addressed through fully automatic and dynamic migration. [See FAQ-Latency]

[Q60] What about Link utilization?

[A60] We will answer this question (below) but first point out that while Conventional Data Centers (CDC's) put a high emphasis on maximizing (or balancing) link flows, the Transaction Fabrix does not need to do that, because: (a) links are cheap¹¹. (b) There is already 2-6 times more link bandwidth available from the servers than a Clos Network, and (c) Our goal is simplicity, resilience and security, not bandwidth (although we have more usable bandwidth (goodput)), or latency (which we have less). It is therefore the wrong concept to focus on the utilization of an abundant resource such as links or bandwidth.

[A60] In a Conventional Data Center *On average*, the core of the data center is the most utilized layer, while the data center edge is lightly utilized [Benson Et al.].

[A60] The core layers in various CDCs have hot-spot links. In some of the data centers, the hot-spots appear only occasionally. In some of the cloud data centers, a significant fraction of core links appear as hot-spots a large fraction of the time.

At the same time, the number of core links that are hot-spots at any given time is less than 25%; (4) Losses are not correlated with links with persistently high utilizations. We observed losses do occur on links with low average utilization indicating that losses are due to momentary bursts; and (5) In general, time-of-day and day-of-week variation exists in many of the datacenters. The variation in link utilization is most significant in the core of the data centers and quite moderate in other layers of the

[Q61] Where do Lambda's come into the picture?

[A61] Lambda's are not what they are touted to be. See: [Serverless is awesome \(if you overlook inflated costs, dislike distributed computing, love vendor lock-in\)](#).

[A61] We treat Lambda's as the next level of entity in the execution hierarchy (Cell, core, process, VM, Container, Lambda). Our layered Labyrinth architecture is general. It works from the highest level, down to the lowest component we can imagine.

¹¹The chip cost for SmartNICs is the same. Many support 8 ports from the same chip – e.g. 4 x 25GbE lanes instead of one 100Gb. The cost of a short copper cable and shielded connector is under \$10 and dropping sharply as volumes increase).